1. Event Naming Convention

All Kafka events across SplitSpend follow a single consistent pattern: **<domain>.<subdomain>.<verb_past_tense>**. The verb is always past tense because an event describes something that has already happened, not a command or a request. Subdomains are included when a domain has distinct sub-contexts that need separate routing — most critically in the Wallet Service, where different consumers must react only to relevant debit types.

Why separate wallet.budget.debited from wallet.main.debited?

The original design emitted a single wallet.debited event for every debit. This created a routing problem: Budget Service consumed it to update daily spend tracking — but it would also fire when a user made a plain Main Balance bank transfer, polluting budget records with non-budget transactions. Splitting into wallet.budget.debited and wallet.main.debited means each consumer subscribes only to events relevant to it.

Why are vendor payments and user-to-user payments internal wallet moves?

When a buyer pays a vendor or another SplitSpend user, both parties are already on the platform. There is no reason to send money out to Paystack and back in again. The Wallet Service simply debits the buyer and credits the recipient in a single atomic ledger operation. Paystack is only involved for deposits (virtual account webhooks) and external bank payouts (Transfer Service).

Event Catalogue

Event	Domain	Subdomain	Verb	Meaning
wallet.credited	wallet	—	credited	Any credit to any wallet balance (deposit, vendor receipt, user-to-user receipt, gift, refund)
wallet.budget.debited	wallet	budget	debited	Budget balance deducted during a spend (vendor or user-to-user payment)
wallet.main.debited	wallet	main	debited	Main balance deducted (fallback spend or external bank transfer pre-debit)
wallet.budget.transfer.completed	wallet	budget.transfer	completed	Main→Budget internal move confirmed; activate budget
wallet.budget.transfer.failed	wallet	budget.transfer	failed	Main→Budget internal move failed; mark budget failed
wallet.main.transfer.initiated	wallet	main.transfer	initiated	Main balance pre-debited; Transfer Service may now call Paystack
wallet.insufficient_funds	wallet	—	insufficient_funds	All balances exhausted; debit cannot proceed
budget.created	budget	—	created	New budget record created (Status = Pending)
budget.activated	budget	—	activated	Wallet confirmed transfer; budget is live
budget.daily.released	budget	daily	released	CRON: daily funds allocated to UserTotalDailyBudget
budget.daily.expired	budget	daily	expired	CRON: unused daily amount returned to Main Balance
budget.completed	budget	—	completed	Budget fully consumed or end date reached

budget.cancelled	budget	—	cancelled	User cancelled budget; funds returned to Main Balance
transfer.created	transfer	—	created	External bank transfer record opened (Pending) — Paystack payout starting
transfer.completed	transfer	—	completed	Paystack confirmed external bank delivery
transfer.failed	transfer	—	failed	Paystack declined or timed out; Wallet reverses the pre-debit
payment.successful	payment	—	successful	Paystack confirmed a deposit into the platform (virtual account webhook)
payment.failed	payment	—	failed	Paystack deposit webhook failed or could not be verified
transaction.created	transaction	—	created	Transaction lifecycle record opened (Pending)
transaction.completed	transaction	—	completed	Full lifecycle success confirmed
transaction.failed	transaction	—	failed	Any step in the chain failed
vendor.payment.requested	vendor	payment	requested	Vendor or user created an in-platform payment request (QR or UserId)
vendor.payment.approved	vendor	payment	approved	Payer approved — triggers internal wallet debit + recipient credit
vendor.payment.rejected	vendor	payment	rejected	Payer declined the request
vendor.payment.expired	vendor	payment	expired	2-minute approval window elapsed with no response
gift.sent	gift	—	sent	Gift budget dispatched to receiver — internal wallet move

2. Observability Strategy

SplitSpend is a distributed microservices system. Observability is built in from day one, not bolted on later.

Three Pillars

Pillar	Tool	What It Gives You
Structured Logging	Serilog + Seq	Searchable, queryable logs per service across the whole system
Distributed Tracing	OpenTelemetry	Follow a single request across every service it touches
Metrics & Monitoring	Azure Application Insights	Live dashboards, alerts, failure rates, response times, dependency health

Structured Logging — Serilog + Seq

Every service writes structured JSON logs with Serilog to two sinks simultaneously: a file sink (rolling log files on disk) and a Seq sink (centralised log server; all 9 services feed into one searchable web UI).

Log Property	Purpose
ServiceName	Identifies which microservice emitted the log
TraceId	Correlates all logs for a single request across all services
UserId	Find every action a specific user triggered
TransactionId	Trace one financial operation end-to-end
EventType	Filter all logs for a specific Kafka event
Level	Information / Warning / Error / Fatal
Environment	“Production” or “Staging” — keeps dev noise out of prod dashboards

Distributed Tracing — OpenTelemetry

A single user action touches 4-5 services. OpenTelemetry propagates a TraceId through every HTTP call and Kafka message, building a complete visual call graph.

- Every ASP.NET Core service instrumented with the OpenTelemetry SDK
- Kafka producer and consumer spans captured automatically
- HTTP calls carry W3C trace context headers between services
- Traces exported to Application Insights for visualisation

Example trace — Vendor payment approval (5 steps):

#	Service	Span	What to Watch
1	Vendor Pay Service	POST /approve	Request received, emits vendor.payment.approved
2	Transaction Service	Kafka consume	Creates transaction (Pending), emits transaction.created
3	Wallet Service	Kafka consume	Debits payer (budget-first), credits vendor Main Balance; emits wallet.budget.debited + wallet.credited
4	Transaction Service	Kafka consume	Consumes wallet.credited — marks transaction Completed, emits transaction.completed
5	Notification Service	Kafka consume	Sends push/SMS to both payer and vendor

Metrics & Monitoring — Azure Application Insights

Feature	What You Can Do
Live Metrics Stream	See requests, failures, and CPU/memory in real time during incidents
Failure Rate Alerts	Get notified instantly if payment.failed events spike above threshold
Dependency Tracking	Auto-tracks Paystack API, SQL Server, Kafka — shows slow or failing dependencies
Application Map	Visual graph of all services and connections — spot bottlenecks at a glance
Custom Dashboards	Pin key metrics: budget creation rate, debit success rate, Kafka consumer lag
Smart Detection	AI anomaly detection flags unusual patterns

Key Alerts (MVP):

Alert Condition	Severity	Response
payment.failed rate > 5% in 5 minutes	Critical	Check Paystack status + Payment Service logs in Seq
wallet.insufficient_funds events spike	Critical	Check Wallet Service for DB lock contention or balance sync errors

Kafka consumer lag > 500 messages	Warning	Scale consumer or investigate slow processing in affected service
Any service HTTP 500 rate > 1%	Warning	Open Application Insights Failures tab; trace to root cause via TraceId
Budget CRON job did not fire by 06:10 AM	Critical	Users miss daily budgets — check Hangfire/Quartz dashboard immediately
transfer.failed events spike	Warning	Check Transfer Service + Paystack payout status; confirm Main Balance reversal completed
vendor.payment.expired rate spikes	Warning	Push notifications may be delayed — check Notification Service consumer lag

Debugging Workflow

1. Check Application Insights Live Metrics — identify which service has elevated failures
2. Open Seq, filter by TraceId or UserId — see the full log chain in seconds
3. Locate the failing span in the OpenTelemetry trace — pinpoint the exact service and operation
4. Check dependency health (Paystack, SQL Server, Kafka) via Application Insights
5. Apply fix, deploy, confirm metrics return to baseline — close the alert

3. Technology Stack

Layer	Technology	Purpose
Backend	ASP.NET Core Web API (C#)	Core business logic & REST APIs
Frontend Web	Angular	Web application
Mobile	.NET MAUI (Android & iOS)	Cross-platform mobile app
Database	SQL Server (MSSQL)	Persistent data storage per service
Payments	Paystack API	Deposits via virtual accounts + external bank transfers (Transfer Service only)
Messaging	Apache Kafka	Async event bus between services
Background Jobs	Hangfire / Quartz.NET	Daily budget release & expiry CRON jobs
Auth	JWT + Refresh Tokens + PIN	User authentication & transaction approval
Logging	Serilog → File + Seq	Structured logs, cross-service search via Seq UI
Tracing	OpenTelemetry	Distributed trace propagation across all services
Monitoring	Azure Application Insights	Live metrics, alerts, dependency tracking, dashboards
Service Discovery	HashiCorp Consul	Service registry, health checks, dynamic routing
API Gateway	ASP.NET Core + YARP	Single entry point: auth, rate limiting, load balancing, circuit breaker
Resilience	Polly	Retry, circuit breaker, timeout, and fallback policies

4. Architecture

4.1 Design Principles

Async — Kafka (Event-Driven)	Sync — REST (Real-Time)
Daily budget processing & release	Paystack deposit processing
Push notifications	Wallet balance checks
Audit / transaction logs	Budget validation before creation

Gift budget events	User authentication
External bank transfer lifecycle	Transfer pre-flight balance check

Architecture Principle: Event-driven core (Kafka) for scalability. REST for critical, time-sensitive operations. Wallet Service is the single source of financial truth — all money moves happen inside it. Vendor and user-to-user payments are purely internal wallet moves; no Paystack involved. Payment Service handles only Paystack deposits (money in). Transfer Service handles only external bank transfers (money out).

4.2 Microservices Overview

#	Service	Responsibility	Criticality
1	Auth Service	Registration, login, JWT, PIN management	High
2	User Service	User profiles, KYC status, roles	High
3	Wallet Service	Sole owner of balances (Main + Budget); all debits, credits, and internal moves	Critical
4	Budget Service	Budget rules, daily splits, gifts, CRON events	Critical
5	Transfer Service	External bank transfers from Main Balance via Paystack Transfers API	High
6	Transaction Service	Transaction lifecycle coordinator for all money movements	High
7	Payment Service	Paystack deposit webhooks only — credits wallet on successful deposit	High
8	Vendor Pay Service	In-platform payment requests (vendor QR / user-to-user); purely internal wallet flow	Medium
9	Notification Service	SMS, email, push across all platform events	Medium

5. Service Discovery & Registration

In a microservices architecture with 9 independent services, each running on its own host or container, services need a reliable way to find each other without hardcoded URLs. HashiCorp Consul provides a centralised service registry, dynamic routing, and continuous health checking.

Why Consul? Without service discovery, every service must know the exact IP and port of every other service it calls. When services scale, restart, or move containers, those hardcoded addresses break. Consul acts as a live phonebook — services register themselves, and callers look up the current healthy address at runtime.

5.1 How It Works

#	Phase	What Happens
1	Register	On startup, each service registers itself with Consul: its name, host, port, and health check endpoint
2	Health Check	Consul polls each service /health endpoint every 10 seconds; unhealthy instances are removed automatically
3	Discover	When Service A needs to call Service B, it queries Consul for the current healthy address — no hardcoded URLs
4	Route	Consul returns one or more healthy instances; the caller connects directly or via a sidecar proxy

5	Deregister	On graceful shutdown, the service deregisters itself; Consul also auto-deregisters instances that fail health checks

5.2 Service Registry

Service Name	Default Port	Health Endpoint	Tags
auth-service	5001	/health	auth, identity, jwt
user-service	5002	/health	user, profile
wallet-service	5003	/health	wallet, finance, critical
budget-service	5004	/health	budget, finance, critical
transfer-service	5005	/health	transfer, paystack, finance
transaction-service	5006	/health	transaction, finance
payment-service	5007	/health	payment, paystack, deposit
pay-service	5008	/health	pay, vendor, user-transfer, internal
notification-service	5009	/health	notification, sms, email, push

6. API Gateway

The API Gateway is the single entry point for all client traffic — web, mobile, and third-party. No client ever calls a microservice directly. Every request flows through the gateway, which handles cross-cutting concerns centrally so individual services stay lean and focused on business logic.

Implementation: Built as a dedicated ASP.NET Core service using YARP (Yet Another Reverse Proxy) — Microsoft's high-performance, production-grade reverse proxy library.

6.1 Responsibilities Overview

Responsibility	Summary
Authentication & Authorisation	Validate JWT tokens and enforce role-based access before any request reaches a service
Rate Limiting	Throttle clients per IP and per user to protect services from abuse and overload
Load Balancing	Distribute traffic evenly across multiple healthy service instances via Consul
Service Discovery	Query Consul at request time to resolve current healthy instances
Circuit Breaker	Use Polly to stop cascading failures when a downstream service is degraded or unreachable
Logging & Monitoring	Emit structured logs via Serilog to Seq; publish metrics to Application Insights
Distributed Tracing	Attach and propagate a CorrelationId on every request across all downstream services
Caching	Cache frequently read, low-volatility responses (e.g. user profiles, bank lists) to reduce downstream load
API Aggregation	Combine responses from multiple services into a single client response where needed

6.2 Authorisation Rules

Route Pattern	Required Role	Notes
/api/auth/*	Public	Login, register, verify — no token required
/api/wallets/*	User	Gateway enforces UserId ownership check

/api/budgets/*	User	Any authenticated user
/api/transfers/*	User + PIN	PIN verified at gateway before routing to Transfer Service
/api/pay/*	User / Vendor	Both users and vendors can create or approve payment requests
/api/admin/*	Admin	Budget cancellation, role assignment
/api/payments/webhook	Internal	Paystack deposit webhook — validated by HMAC-SHA512 signature only, no JWT
/api/transfers/webhook	Internal	Paystack transfer webhook — HMAC-SHA512 only, no JWT

6.3 Rate Limiting

Policy	Limit	Window	Applied To
Global IP Limit	100 req	60 seconds	All unauthenticated requests
Authenticated User	300 req	60 seconds	Per UserId across all protected routes
Payment Endpoints	10 req	60 seconds	/api/payments/webhook — prevent webhook flooding
Transfer Endpoints	5 req	60 seconds	/api/transfers/* per UserId — tighter limit on external money movement
Auth Endpoints	5 req	60 seconds	/api/auth/login — brute-force protection
Pay Endpoints	20 req	60 seconds	/api/pay/* per UserId — covers vendor and user-to-user requests

6.4 Aggregated Endpoints

Aggregated Route	Sources	Client Use Case
GET /api/dashboard/{userId}	Wallet Service (balances) + Budget Service (daily summary) + Transaction Service (last 5 transactions)	Home screen — shows balance, daily budget remaining, and recent activity in one call
GET /api/pay/{id}/detail	Pay Service (request details) + User Service (requester profile) + Wallet Service (payer balance check)	Payment approval screen — shows full request context before payer approves